

Compléments d'Algorithmique

Chapitre 4 : Itérateurs, Générateurs et Flux de données

Hadrien Mélot
Service d'Algorithmique
Département d'Informatique
Faculté des Sciences



Chapitre 4. Itérateurs, Générateurs et Flux de données

- 1 Encapsulation
- 2 Itérateurs
- 3 Générateurs
- 4 Flux IO
- 5 Illustration : analyse de code ADN

Encapsulation

Encapsulation

Au chapitre présent, nous avons défini la classe `Node` comme ceci.

Définition de classe

```
class Node:
    def __init__(self, data = 0, next = None):
        self.data = data
        self.next = next

    def __str__(self):
        return str(self.data)

    def __repr__(self):
        return repr(self.data) + ' -> ' + repr(self.next)

    def set_next(self, next):
        self.next = next

    def get_next(self):
        return self.next

    def set_data(self, data):
        self.data = data

    def get_data(self):
        return self.data
```

Encapsulation

Pourquoi avoir défini des méthodes `set` et `get` pour les champs de la classe `Node`, alors que l'on peut y accéder directement ?

C'est le principe d'**encapsulation**, qui consiste à cacher les détails d'implémentation de la classe à l'utilisateur de celle-ci.

- Dans certains langages, comme Java, les champs et les méthodes peuvent être déclarés :
 - ▶ **public** : accessibles directement ;
 - ▶ **protected** : accessibles par la classe elle-même et ses classes filles ;
 - ▶ **private** : accessibles uniquement par la classe elle-même.
- En Python, il n'y a pas de notion de champs privés, mais on peut simuler l'encapsulation par convention.

Encapsulation

L'**encapsulation par convention** en Python consiste à préfixer les noms des champs ou des méthodes par

- un underscore (`_`) pour indiquer qu'ils ne devraient pas être accédés directement en dehors de la classe et de ses filles (**protected**);
- un double underscore (`__`) pour indiquer que le champ ou la méthode est privé (**private**).

Exemple

```
class Person:
    def __init__(self, nom, age):
        self._nom = nom # protected
        self.__age = age # private

    def __get_sensitive_data(self):
        return self.__age
```

Encapsulation

Il s'agit d'une **convention** dans le sens où rien n'empêche de la contourner : les champs et les méthodes **protected** sont toujours accessibles.

Exemple

```
p = Person("Alice", 25)
print(p._nom)
```

Résultat

```
Alice
```

Remarque : l'encapsulation par convention est une aide à la lecture du code, mais ne garantit donc pas la protection des champs ou des méthodes. Ceci dit, il est recommandé de respecter le statut (ici **protected**) voulu par les développeurs.

Encapsulation

Les champs ou méthodes privés (__) sont plus difficiles à accéder directement (en dehors de la classe elle même), mais cela reste possible comme on le verra ci-après.

Exemple

```
print(p.__age)
x = p.__get_sensitive_data()
```

Résultat

```
AttributeError: 'Person' object has no attribute '__age'
AttributeError: 'Person' object has no attribute '__get_sensitive_data'
```


Encapsulation

En réalité, Python renomme en interne les champs ou méthodes privés en ajoutant le nom de la classe devant le nom original (cela s'appelle le **mangling**).

Mais donc, si on veut vraiment accéder à un champ privé, cela reste possible (**mais n'est pas recommandé**).

Exemple

```
print(p._Person__age)
x = p._Person__get_sensitive_data()
print(x)
```

Résultat

25

25

Encapsulation

Une autre forme d'encapsulation : déclarer une **classe interne**.

Une classe interne permet de “cacher” une classe à l'extérieur de la classe qui la contient. Par exemple, nous pourrions rendre `Node` interne à la classe `ListeChaine`.

Refactorisons le code de nos structures de données :

- `Node` est une classe interne de `ListeChaine` ;
- les champs de `Node` sont protégés ;
- les méthodes de `Node` sont supprimées (elles ne sont plus nécessaires, on va accéder directement au champs, car la classe est interne).
- les méthodes de `ListeChaine` (qui sont destinées à être utilisées par des classes filles) sont protégées.

Encapsulation

Définition de classe (partiel)

```
class ListeChaine:
    class Node:
        def __init__(self, data=0, next=None):
            self._data = data
            self._next = next
        def __repr__(self):
            return repr(self._data) + ' -> ' + repr(self._next)

    def __init__(self):
        self._n = 0
        self._first = None
        self._last = None
    def _insert_first(self, data):
        self._first = self.Node(data, self._first)
        if self._last is None:
            self._last = self._first
        self._n += 1
    (...)
```

Le code complet se trouve sur Moodle.

Encapsulation

Lors de l'affichage de la documentation d'une classe (via `help` par exemple), les éléments `protected` et `private` ne sont pas affichés.

Encore une fois, même si, en Python, ce n'est qu'une convention, cela montre bien que le but est de cacher ces éléments.

(voir démonstration en séance, pour la documentation
des classes `ListeChaine`, `Pile` et `File`)

Itérateurs

Itérateurs

Les Piles et les Files que nous avons conçues ne sont pas itérables.

Essai

```
p = Pile()
p.push(1)
p.push(2)
p.push(3)
for i in p:
    print(i)
```

Résultat

```
for i in p:
    ^
TypeError: 'Pile' object is not iterable
```

Itérateurs

Pourtant, comme pour toute structure de données, il est utile de pouvoir parcourir les éléments via une boucle `for`, ou d'utiliser les fonctions comme `map` ou `filter`, qui s'appliquent sur des objets itérables.

Nous allons remédier à cela. Explorons d'abord la notion d'**itérateur**.

Itérateurs

La fonction `iter` retourne un objet **itérateur** et prend en argument une séquence ou un objet itérable.

On utilise la fonction `next` pour accéder aux éléments de l'itérateur. Chaque appel permet d'accéder à l'élément suivant.

Exemple

```
t = [1, 2, 3]
it = iter(t)
print(it)
print(next(it))
print(next(it))
```

Résultat

```
<list_iterator object at 0x10b1467d0>
1
2
```


Itérateurs

Si l'on va trop loin, une exception de type `StopIteration` est levée.

Exemple

```
t = [1, 2, 3]
it = iter(t)
print(next(it))
print(next(it))
print(next(it))
print(next(it))
```

Résultat

```
1
2
3
Traceback (most recent call last):
  (...)
StopIteration
```

Itérateurs

Cela ne se voit pas dans une boucle `for`, ou dans d'autres mécanismes sur les itérables (comme `map`) car Python gère automatiquement l'exception.

Exemple

```
t = [1, 2, 3]
for i in iter(t):
    print(i)
```

Résultat

```
1
2
3
```

Et comme on l'a déjà vu de nombreuses fois, il n'est pas nécessaire d'utiliser `iter` explicitement. Si un objet est itérable, Python le transforme automatiquement en itérateur là où c'est approprié.

Itérateurs

Cela ne marche toujours pas pour nos piles, car elles ne sont pas itérables.

Essai (suite)

```
i = iter(p)
```

Résultat

```
TypeError: 'Pile' object is not iterable
```

Comment rendre un objet itérable ?

Nous allons créer un **itérateur** pour nos listes chaînées (et donc, par héritage, pour nos piles et nos files), sous la forme d'une **classe interne**.

Celui-ci doit :

- Définir les variables d'instance nécessaires pour connaître la “position courante” dans l'itération ;
- Implémenter `__iter__` qui doit retourner l'objet itérable lui-même ;
- Implémenter `__next__` qui doit retourner l'élément courant (ou lancer `StopIteration`) après avoir mis à jour les variables pour préparer l'itération suivante.

Comment rendre un objet itérable ?

Définition de classe (mise à jour)

```
class ListeChaine:  
    (...)  
    class ListeChaineIterator:  
        def __init__(self, first):  
            self._current = first  
  
        def __iter__(self):  
            return self  
  
        def __next__(self):  
            if self._current is None:  
                raise StopIteration  
            data = self._current._data  
            self._current = self._current._next  
            return data
```

Comment rendre un objet itérable ?

Nous allons également ajouter une méthode `__iter__` dans `ListeChaine` pour retourner un `ListeChaineIterator` automatiquement quand on itérera sur nos listes chaînées.

Définition de classe (mise à jour)

```
class ListeChaine:  
    (...)  
    def __iter__(self):  
        return self.ListeChaineIterator(self._first)
```

Comment rendre un objet itérable ?

Utilisation

```
a = ListeChaine()
a._insert_first(1)
a._insert_first(2)
a._insert_first(3)

for i in a:
    print(i)
```

Résultat

```
3
2
1
```

Comment rendre un objet itérable ?

Grâce à l'héritage, nos piles sont devenues itérables !

Utilisation

```
p = Pile()
p.push(1)
p.push(2)
p.push(3)

i = iter(p)
print(next(i))
print(next(i))

for item in p:
    print(item, end=' ')
print()
```

Résultat

```
3
2
3 2 1
```


Comment rendre un objet itérable ?

Grâce à l'héritage, nos files sont aussi itérables, sans travail supplémentaire.

Utilisation

```
f = File()
f.insert(1)
f.insert(2)
f.insert(3)

for item in f:
    print(item, end=' ')
print()
```

Résultat

```
1 2 3
```

Comment rendre un objet itérable ?

Le fait d'avoir créé un itérateur sous la forme d'une **classe interne** (plutôt que de rendre `ListeChaine` elle-même itérable) permet de faire des boucles imbriquées sur des listes chaînées sans interférer les unes avec les autres : à chaque boucle, un itérateur indépendant est créé.

Exemple

```
p = Pile()
p.push(1)
p.push(2)
p.push(3)

for i in p:
    for j in p:
        print(i, j)
```

Comment rendre un objet itérable ?

Résultat

3 3
3 2
3 1
2 3
2 2
2 1
1 3
1 2
1 1

Opérateurs `[]` et `[] =`

En réalité, si nous n'avions pas rendu nos SD itérables, mais que nous avions surchargé l'opérateur `[]` (via la méthode spéciale `__getitem__`), Python aurait créé un itérateur par défaut basé sur cette méthode.

Commençons par surcharger `__getitem__` et `__setitem__` pour accéder et modifier les éléments d'une structure.

Opérateurs [] et [] =

Définition de classe (mise à jour)

```

class ListeChaine:
    (...)
    def __getitem__(self, index):
        if index < 0 or index >= self._n:
            raise IndexError("Index out of range")
        p = self._first
        for _ in range(index):
            p = p._next
        return p._data

    def __setitem__(self, index, value):
        if index < 0 or index >= self._n:
            raise IndexError("Index out of range")
        p = self._first
        for _ in range(index):
            p = p._next
        p._data = value

```

Les complexités sont en $O(n)$ (pire des cas : indice = $n - 1$).

Opérateurs [] et [] =

Utilisation

```
p = Pile()
p.push(1)
p.push(2)
p.push(3)
print(p[1])
p[1] = 5
print(p)
```

Résultat

```
2
| 3 |
| 5 |
| 1 |
-----
```

Opérateurs [] et [] =

Et encore une fois, grâce à l'héritage :

Utilisation

```
f = File()
f.insert(1)
f.insert(2)
f.insert(3)
print(f[2])
f[0] = 42
print(f)
```

Résultat

```
3
42 <- 2 <- 3
```

Itérateur par défaut

Supposons que nous **commentions** `__iter__` et `__next__`.

Si `__getitem__` existe, Python va créer un **itérateur par défaut** basé sur celui-ci : à chaque `next`, il va incrémenter un indice qu'il maintient automatiquement et faire appel à `__getitem__`.

(voir démonstration en séance)

Cela fonctionne, mais il est préférable de définir explicitement les itérateurs pour maîtriser la complexité et le comportement.

Itérateur par défaut

En effet, avec l'itérateur par défaut, chaque itération fait appel à `__getitem__`, et donc a une complexité en $O(n)$, au lieu de la complexité en $O(1)$ que nous avons obtenue en implémentant `__next__` nous même et en maintenant une référence vers le noeud courant.

Cela signifie qu'une boucle `for` sur une pile ou une file est en $O(n^2)$ au lieu de $O(n)$ (si le corps de la boucle est en $O(1)$).

(voir comparaison de performance en séance)

Nous conserverons donc à la fois `__getitem__` / `__setitem__` pour accéder et modifier une valeur via son indice, mais aussi `__iter__` / `__next__` pour itérer efficacement sur nos piles et files.

Itérateur pour éviter de stocker une séquence en mémoire

Pour créer un compteur aussi grand que l'on veut, on pourrait utiliser une très grande liste d'entiers, mais cela prendrait de la mémoire.

Avec le concept d'itérateur, on peut créer un objet qui génère les entiers à la demande, sans stocker la séquence en mémoire (ce n'était pas le but de nos structures de données qui elles stockent les éléments).

Itérateur pour éviter de stocker une séquence en mémoire

Définition de classe

```
class Compteur:
    def __init__(self, limite):
        self._limite = limite
        self._compteur = 1

    def __iter__(self):
        return self

    def __next__(self):
        if self._compteur <= self._limite:
            valeur = self._compteur
            self._compteur += 1
            return valeur
        else:
            raise StopIteration
```

Itérateur pour éviter de stocker une séquence en mémoire

Utilisation

```
c = Compteur(5)
for nombre in c:
    print(nombre, end = ' ')
```

Résultat

```
1 2 3 4 5
```

Donc, une bonne utilisation des itérateurs peut permettre de gagner en performance mais aussi en mémoire.

Générateurs

Générateurs

En Python, un **générateur** est un type spécial d'itérateur qui simplifie la création de ceux-ci.

- On implémente un générateur comme une fonction qui contient un ou plusieurs **mots-clés** `yield` (à la place de `return`) ;
- Un `yield` permet de "suspendre" l'exécution et de la reprendre plus tard ;
- Il ne nécessite pas d'implémenter `__iter__` et `__next__`.

Générateurs

Exemple

```
def compteur(limite):  
    compteur = 1  
    while compteur <= limite:  
        yield compteur  
        compteur += 1  
  
for nombre in compteur(5):  
    print(nombre, end=' ')
```

Résultat

1 2 3 4 5

Techniquement, les générateurs peuvent provoquer des boucles infinies (si nous n'avons pas mis de limite).

Générateurs

Exemple de générateur infini, et gestion via une boucle limitée.

Exemple

```
def fibonacci():  
    a, b = 0, 1  
    while True:  
        yield a  
        a, b = b, a + b  
  
gen = fibonacci()  
for _ in range(10):  
    print(next(gen), end = ' ')
```

Résultat

0 1 1 2 3 5 8 13 21 34

Générateurs

Un générateur peut utiliser la récursivité.

Exemple : générateur de permutations.

- **générateur** : `permutations(elements, current=[])` ;
- `elements` est la liste des éléments restants à permuter ;
- `current` est la permutation courante ;
- **Cas récursif** : pour tout `elements[i]`, on enlève cet élément que l'on place à la fin de `current`, puis on génère (`yield`) les permutations des éléments restants obtenus récursivement ;
- **Cas de base** : si `elements` est vide, on a une permutation complète, on la `yield`.

Générateurs

Exemple

```
def permutations(elements, current=[]):
    if len(elements) == 0:
        yield current
    else:
        for i in range(len(elements)):
            for p in permutations(elements[:i] + elements[i+1:], \
                                   current + [elements[i]]):
                yield p

for p in permutations([1, 2, 3]):
    print(p)
```

Le cas récursif “yield” ce qui est retourné par un sous-générateur (récursif). En conséquence, seules des permutations complètes sont générées (grâce au case de base). C’est l’équivalent du `return` récursif pour les fonctions classiques.

Générateurs

Résultat

[1, 2, 3]

[1, 3, 2]

[2, 1, 3]

[2, 3, 1]

[3, 1, 2]

[3, 2, 1]

Ce générateur produit les permutations **une par une**, sans stocker toutes les permutations en mémoire !

Différences en Itérateurs et Générateurs

Caractéristique	Itérateur	Générateur
Définition	Classe avec <code>iter / next</code>	Fonction avec <code>yield</code>
Mémoire	Peut stocker des données	Génère à la volée
Complexité	Plus complexe à écrire	Plus simple et lisible
Utilisation	Requiert une classe dédiée	Fonction simple et concise
Gestion de l'état	Stocké dans des attributs	Géré automatiquement

Flux IO

Flux IO

Les **flux IO** sont des objets qui permettent de lire ou écrire des données.

- **IO** signifie **Input / Output** (entrée / sortie).
- Les flux d'**entrée** sont des objets qui permettent de **lire** des données depuis un fichier, le clavier, le réseau, etc.
- Les flux de **sortie** sont des objets qui permettent d'**écrire** des données vers un fichier, l'écran, le réseau, etc.
- Les flux sont des objets itérables.

Flux IO

Types de flux :

- Fichiers : lecture et écriture de fichiers texte et binaires.
- Flux standards (clavier et terminal, voir ci après).
- Sockets : communication réseau (non couvert dans ce cours).
- Pipelines : chaîne de traitement où la sortie d'un processus sert directement d'entrée au suivant (non couvert dans ce cours).

Les flux utilise des zones mémoires temporaires appelées **buffers** (ou mémoire tampon) pour stocker les données en attente d'être traitées. On peut contrôler le buffering comme on le verra ci-après.

Fichiers

Vous avez déjà utilisé des flux fichiers.

Exemple

```
f = open("fichier.txt", "r")
for ligne in f:
    print(ligne, end='')
f.close()
```

- `open` ouvre un fichier (ici, en lecture via `"r"`);
- `f` est un flux d'entrée "fichier".
- On peut itérer sur `f` pour lire les lignes du fichier.
- `close` ferme le fichier.

Fichiers

On peut aussi utiliser un bloc `with open(...)` as `f`: qui ferme automatiquement le fichier à la fin du bloc.

Exemple

```
with open("fichier.txt", "r") as f:
    for ligne in f:
        print(ligne, end='')
```

Le fichier est fermé automatiquement à la fin du bloc. C'est la manière recommandée d'ouvrir un fichier.

On peut aussi

- ouvrir un fichier en écriture ("`w`") ou en ajout ("`a`") et écrire les données avec `write` ou `writelines`;
- au besoin, on peut spécifier l'encodage (e.g., `encoding="utf-8"`).

Flux standards

Python utilise trois flux standards accessibles via le module `sys`.

- `sys.stdin` : flux d'entrée standard (clavier) ;
- `sys.stdout` : flux de sortie standard (terminal) ;
- `sys.stderr` : flux d'erreurs standard (terminal).

On peut les utiliser comme des fichiers (tous les types de flux ont une interface commune, via l'héritage des objets IO).

Parfois, il peut être utile de rediriger ces flux vers un fichier ou un autre flux.

Flux standards

Exemple

```
import sys

# Lecture depuis l'entrée standard
nom = input("Entrez votre nom : ")

# Ecriture vers la sortie standard
print(f"Bonjour, {nom} !")

# Sortie standard, comme un fichier
sys.stdout.write("Ceci est un message via sys.stdout\n")

# Affichage d'une erreur sur stderr
sys.stderr.write("Ceci est un message d'erreur\n")
```

Flux standards

Exemple (suite)

```
# Rediriger le flux de sortie standard vers un fichier
with open("output.txt", "w") as f:
    sys.stdout = f
    print("Ce texte sera écrit dans le fichier output.txt")

# Rétablir la sortie standard
# (sinon print ne fonctionne plus car f est fermé après le bloc with)
sys.stdout = sys.__stdout__

# Rediriger le flux d'erreur standard vers un fichier
with open("errors.log", "w") as f:
    sys.stderr = f
    sys.stderr.write("Ceci est un message écrit dans error.log\n")

# Rétablir la sortie d'erreur standard
sys.stderr = sys.__stderr__
```

Buffering

Itérer sur un fichier via une boucle `for` permet de lire les lignes une par une, ce qui est utile pour des gros fichiers : tout n'est pas chargé en une fois en mémoire. Comme on l'a vu précédemment, c'est un des avantages des itérateurs.

Ce n'est pas le cas de la méthode `read` utilisée sans argument : elle charge tout le fichier en mémoire.

Exemple

```
with open("fichier.txt", "r") as f:  
    all_content = f.read()
```

Buffering

Un **buffer** est une zone de mémoire temporaire qui stocke les données en attente d'être traitées.

Quand on itère sur un fichier en lecture, le mode de buffering (ou "mise en mémoire tampon") par défaut est le buffering ligne par ligne. On peut spécifier le mode de buffering via l'argument `buffering` de la méthode `open`.

- Le buffering contrôle la manière dont les entrées/sorties sont gérées.
- Il optimise les performances en réduisant les appels système.
- Python propose plusieurs modes de buffering configurables.

Pas de Buffering (buffering = 0)

- Disponible uniquement pour les fichiers binaires ("rb", "wb");
- Chaque opération est directement transmise au système ;
- Moins efficace mais utile pour certaines applications bas niveau.

Exemple

```
with open("file.bin", "wb", buffering=0) as f:  
    f.write(b"Hello")
```

Nous n'allons pas nous y étendre, car nous utiliserons principalement des fichiers textes.

Buffering en mode Ligne (buffering = 1)

- Disponible pour les fichiers textes ("r", "w").
- Ecriture dès qu'un saut de ligne (`\n`) est rencontré.
- Utile pour les journaux (fichiers log) et les sorties interactives.

Exemple

```
with open("file.txt", "w", buffering=1) as f:  
    f.write("Hello\n") # Ecrit immédiatement  
    f.write("World")   # Reste en mémoire
```


Buffering en mode Bloc (buffering > 1)

- Un buffer de taille spécifiée (exprimée en octets) stocke temporairement les données.
- Les données sont écrites lorsqu'il est plein.
- Plus performant pour les fichiers volumineux.

Exemple

```
with open("file.txt", "w", buffering=8192) as f:  
    f.write("Hello World")
```

Buffering automatique (buffering = -1)

- Mode par défaut en Python.
- En mode texte : ligne si sortie terminal, bloc sinon (par exemple, dans un fichier).
- En mode binaire : bloc.

Exemple

```
with open("file.txt", "w") as f: # buffering = -1 par défaut
    f.write("Hello")
```

Seek & Tell

Une autre manière de ne pas charger tout le fichier en mémoire est de se déplacer dans le fichier, et d'utiliser `read` en lui spécifiant le nombre de caractères à lire.

Exemple :

- lire 10 caractères : `read(10)` ;
- se déplacer dans le fichier à une position `x` donnée : `seek(x)` ;
- obtenir la position courante : `tell()` ;

Une position est l'indice d'un caractère dans le fichier, le premier caractère étant à la position 0.

Seek & Tell

Contenu d'un fichier fichier.txt

abcdefghijklmnopqrstuvwxyz

Exemple

```
with open("fichier.txt", "r") as f:
    f.seek(3)
    print(f.read(10))
    print(f.tell())
    f.seek(25)
    print(f.read(1))
```

Résultat

```
defghijklm
13
z
```

Illustration : analyse de code ADN

Illustration : analyse de code ADN

Le code ADN est composé de 4 bases : A, C, G et T (les nucléotides).

Ces nucléotides sont associées par paires dans les génomes. En bioinformatique, pour analyser des séquences ADN, il n'est pas nécessaire de stocker toutes les paires puisque chaque base est toujours associée à la même autre base. On peut donc représenter un génome sous la forme d'une longue chaîne de caractères ne contenant que des A, C, G et T.

Illustration : analyse de code ADN

Le génome humain contient environ 3,2 milliards de paires de bases. Dans un fichier, chaque caractère est stocké sur 1 octet, donc un fichier qui contiendrait le génome humain aurait la taille suivante :

$$3,2 \times 10^9 \text{ octets} \simeq 2.98 \text{ Gb.}$$



Remarque : En stockant le génome différemment, par exemple au format binaire où chaque octet contiendrait 4 nucléotides, on pourrait réduire cela à environ 800 Mo.

Illustration : analyse de code ADN

Nous allons écrire

- un script qui crée une séquence de nucléotides aléatoires et l'écrit dans un fichier (qui contiendra une seule ligne avec une très longue chaîne de caractères) ;
- un script qui lit ce fichier et cherche les positions d'une sous-séquence donnée.
- nous allons tracer la mémoire RAM utilisée et tester en faisant varier la taille d'un buffer pour lire le fichier.

Présentation du code en séance (disponible sur Moodle)